COMP47350: Data Analytics (Conv)

Lecture 10: Data Transformations

Dr. Georgiana Ifrim
georgiana.ifrim@ucd.ie

Insight Centre for Data Analytics
School of Computer Science
University College Dublin

2025/26

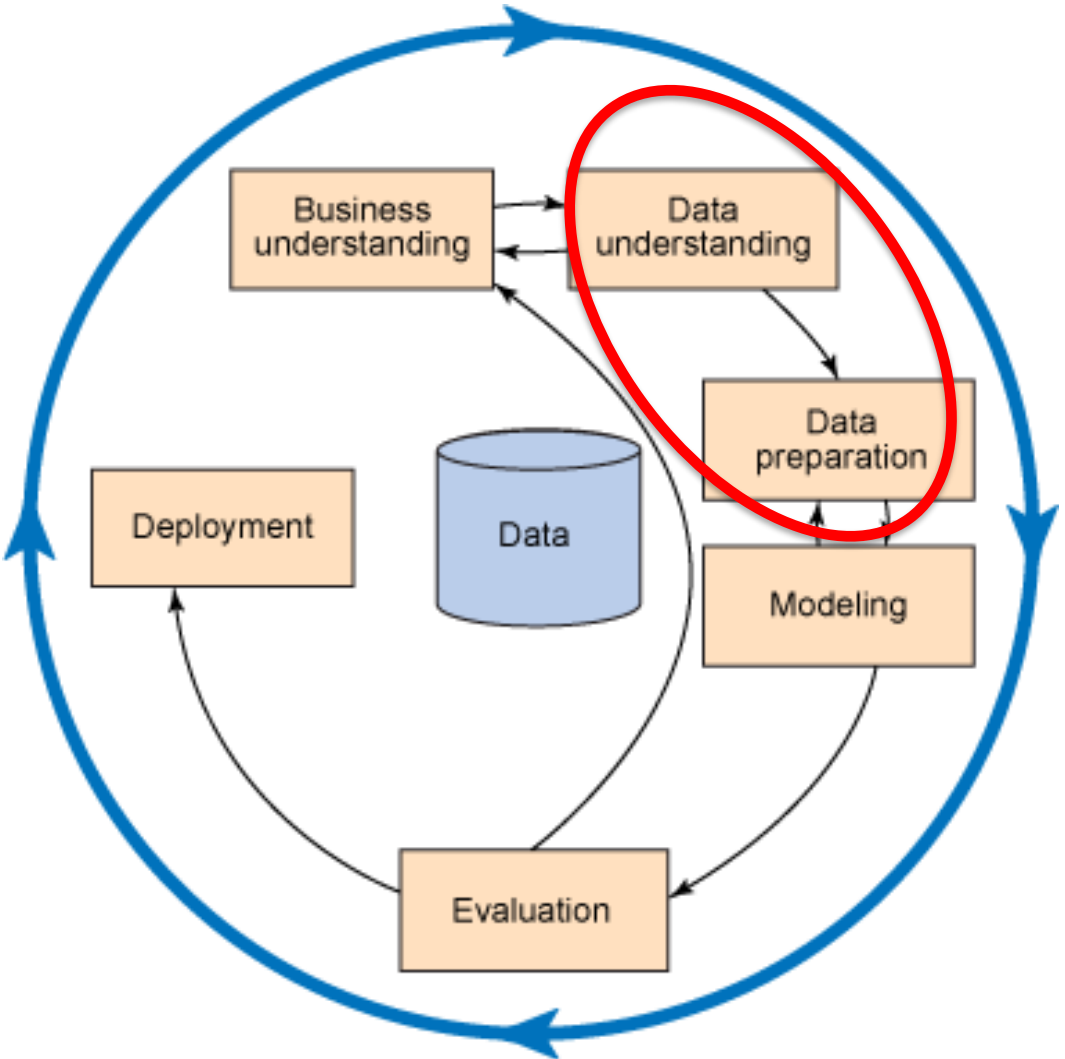
Module Topics

- **Python Environment** (Anaconda, Jupyter Notebook)
- **Getting Data** (Web scrapping, APIs, DBs)
- **Understanding Data** (slicing, visualisation)
- **Preparing Data** (cleaning, transformation)
- **Modeling & Evaluation** (machine learning)

Data Analytics Project Lifecycle:

CRISP-DM

CRISP-DM: **CR**oss-**I**ndustry **S**tandard **P**rocess for **D**ata **M**ining



Data Preparation

- **Data Transformations:** Changing the original data to make it easier to find patterns and make it more compatible with some machine learning algorithms

- These transformations can allow us to find better predictive patterns in the data
  - **Feature Scaling:**
    - **Normalization**
    - **Binning**
  - **Sampling of instances**
    - **Random, stratified, over-sampling**

Basketball ABT

ID	Position	Height	Weight	Career Stage	Age	Sponsorship Earnings	Shoe Sponsor
1	forward	192	218	veteran	29	561	yes
2	center	218	251	mid-career	35	60	no
3	forward	197	221	rookie	22	1,312	no
4	forward	192	219	rookie	22	1,359	no
5	forward	198	223	veteran	29	362	yes
6	guard	166	188	rookie	21	1,536	yes
7	forward	195	221	veteran	25	694	no
8	guard	182	199	rookie	21	1,678	yes
9	guard	189	199	mid-career	27	385	yes
10	forward	205	232	rookie	24	1,416	no
11	center	206	246	mid-career	29	314	no
12	guard	185	207	rookie	23	1,497	yes
13	guard	172	183	rookie	24	1,383	yes
14	guard	169	183	rookie	24	1,034	yes
15	guard	185	197	mid-career	29	178	yes
16	forward	215	232	mid-career	30	434	no
17	guard	158	184	veteran	29	162	yes
18	guard	190	207	mid-career	27	648	yes
19	center	195	235	mid-career	28	481	no
20	guard	192	200	mid-career	32	427	yes
21	forward	202	220	mid-career	31	542	no
22	forward	184	213	mid-career	32	12	no
23	forward	190	215	rookie	22	1,179	no
24	guard	178	193	rookie	21	1,078	no
25	guard	185	200	mid-career	31	213	yes
26	forward	191	218	rookie	19	1,855	no
27	center	196	235	veteran	32	47	no
28	forward	198	221	rookie	22	1,409	no
29	center	207	247	veteran	27	1,065	no
30	center	201	244	mid-career	25	1,111	yes

Normalization of features = changing the feature values to fall into a given range (for continuous features)

Normalization

- Important to make feature-values comparable: features can have very different units such as: Age (years) in range [18, 90], Height (cm) in range [150, 230], Salary (euro) in range [50000, 100000]

- Can lead to improvements in the learning model (when applied to un-normalized vs normalized data). Many ML models benefit from normalizing the features (distance-based algorithms like K-Nearest-Neighbor and Neural Networks)

- Need to apply this transformation to input features in both training and test data

- The target feature is generally left as is (no need to scale the target)

Normalization

- **Range normalization:**
  - Convert a feature **a** into new feature **a'** with values in range [0,1]
  - Subtract the min value for that feature, and divide by the range (max – min)

$$a_i' = \frac{a_i - min(a)}{max(a) - min(a)}$$

- Sensitive to outliers in the dataset (i.e., extreme min or max values for the feature)
- Normalized feature loses the original units of the data

Normalization

- Another way to normalize features: **Standardization**
- To calculate a **standard score (aka z-score)** we subtract the mean and divide by the standard deviation for that feature

$$a_i' = \frac{a_i - \bar{a}}{sd(a)}$$

- The new standardized feature has a mean of 0 (data is centered at mean 0) and standard deviation of 1.
- This transformation is very common and can help some learning algorithms.

Normalization

- **Standard score (aka z-score):** measures how many standard deviations away from the mean is the feature value
- In our example, a height of **206 = mean + 1.622 *stdev**
  = 193 + 1.622 * 8.26

$$a_i' = \frac{a_i - \bar{a}}{sd(a)}$$

Z-score

- Advantage: makes feature values comparable; more robust to outliers than min-max normalisation
- Disadvantage: loses original meaning/scale of features

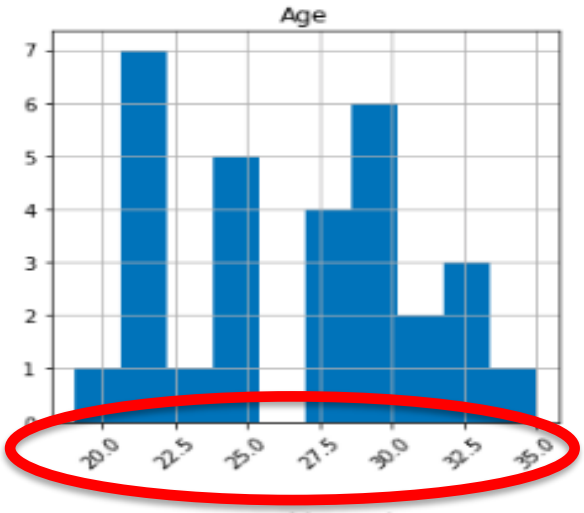
Basketball ABT

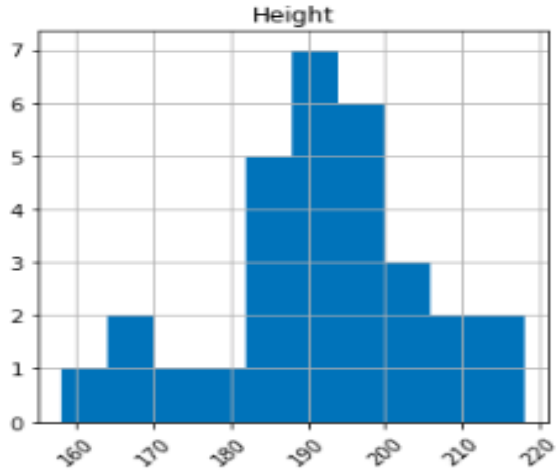
Range normalization and **standardization** example for **10 samples** of feature **Height** and **Sponsorship Earnings**

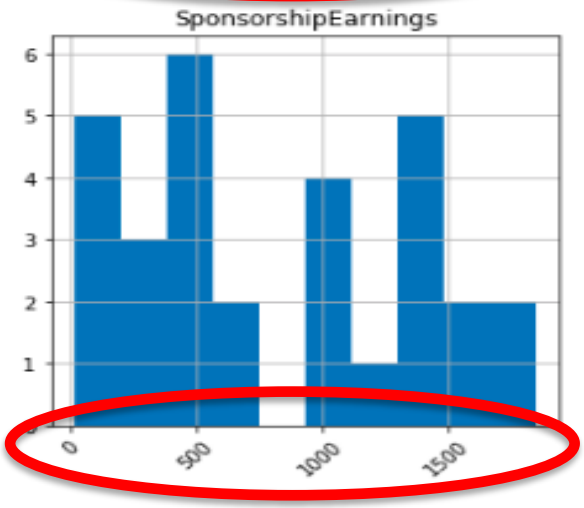
	Height			Sponsorship Earnings		
	Values	Range	Standard	Values	Range	Standard
	192	0.500	-0.073	561	0.315	-0.649
	197	0.679	0.533	1,312	0.776	0.762
	192	0.500	-0.073	1,359	0.804	0.850
	182	0.143	-1.283	1,678	1.000	1.449
	206	1.000	1.622	314	0.164	-1.114
	192	0.500	-0.073	427	0.233	-0.901
	190	0.429	-0.315	1,179	0.694	0.512
	178	0.000	-1.767	1,078	0.632	0.322
	196	0.643	0.412	47	0.000	-1.615
	201	0.821	1.017	1111	0.652	0.384
Max	206			1,678		
Min	178			47		
Mean	193			907		
Std Dev	8.26			532.18		

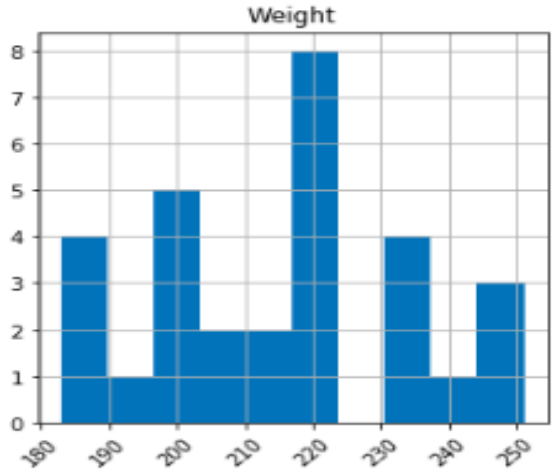
Basketball ABT

Histograms of numeric columns for **original data** **in Basketball dataset**



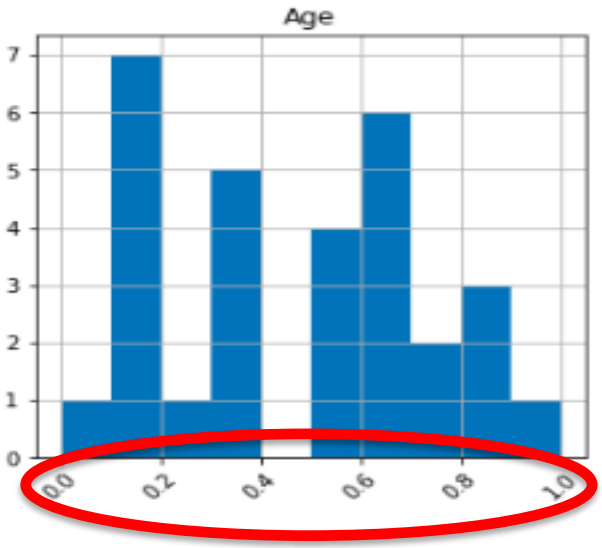


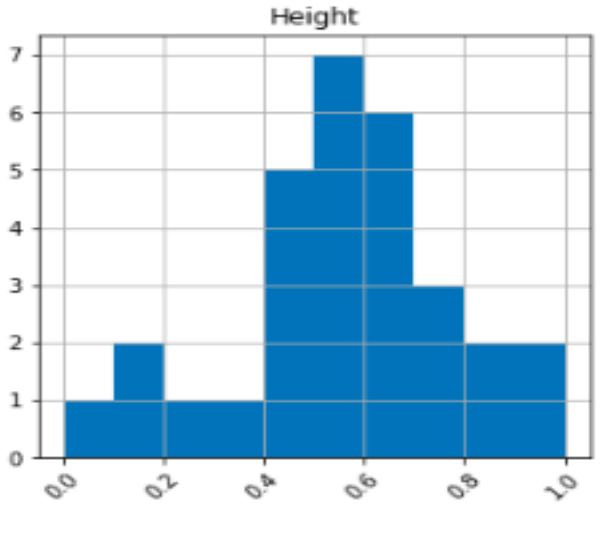


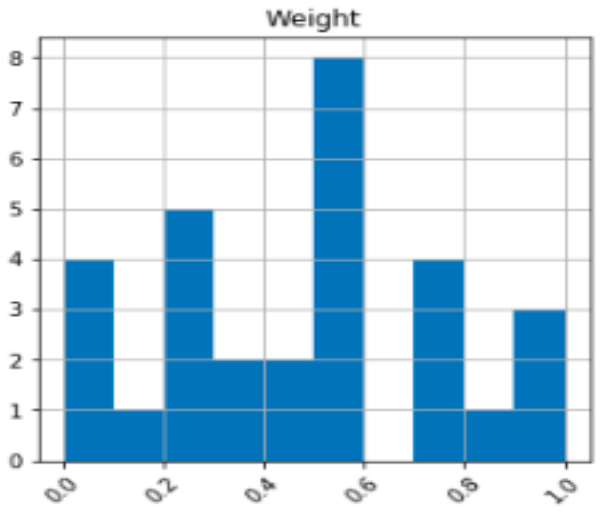


Basketball ABT

Histograms of numeric columns for **range normalized** **data**

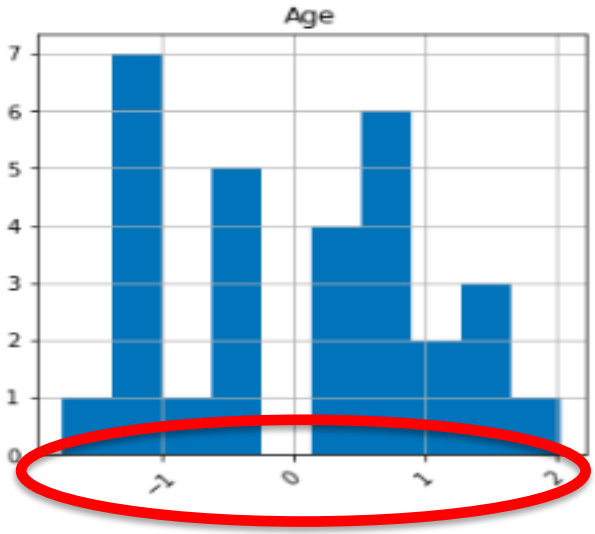


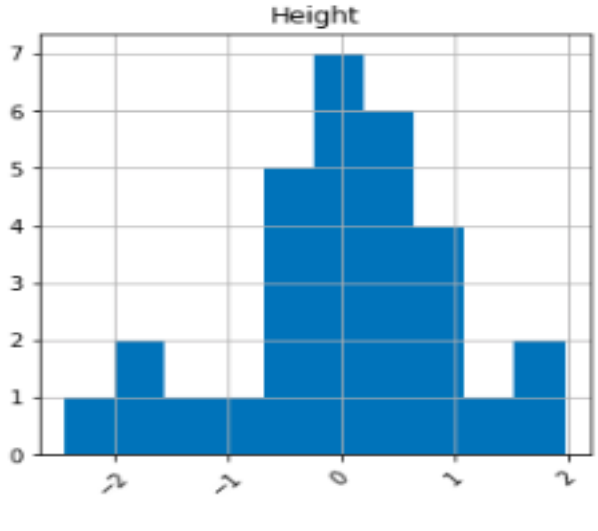


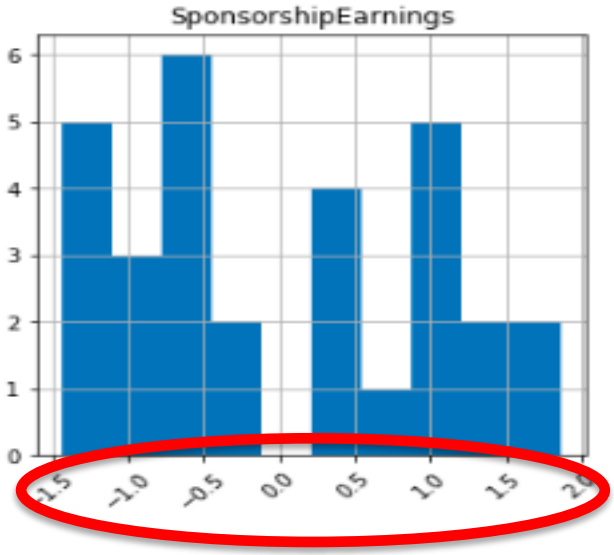


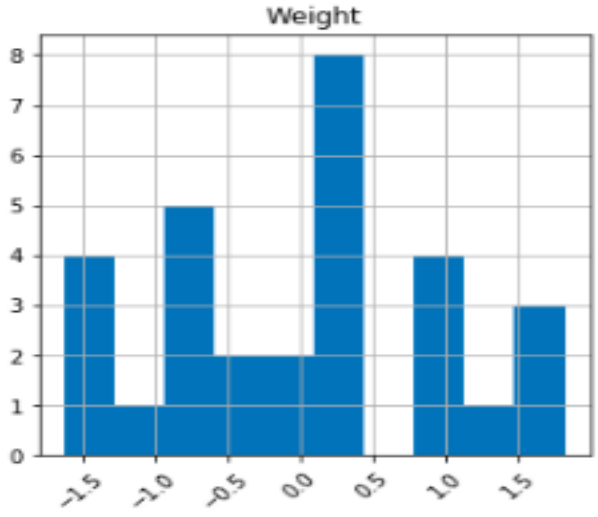
Basketball ABT

Histograms of numeric columns for **standardized** **data**









Binning of features = discretizing numeric data (for continuous features)

Binning

- **Binning:** converting a continuous feature into a categorical feature
- Why? Reduce the number of values a feature can take; **can help reduce the effect of outliers**

- Define a series of ranges (called **bins**) for the continuous feature, that correspond to the levels of the new categorical feature (e.g., levels are 0-20k, 20k-40k, etc.)

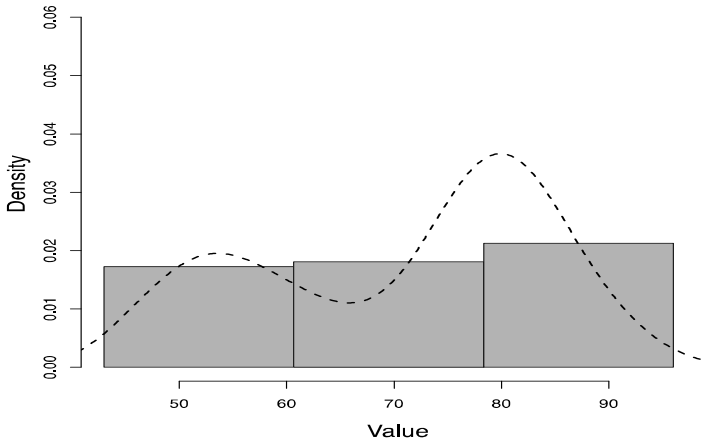
- Two popular ways of defining bins:
  - **equal-width binning** (bins have fixed width; our default option for histograms, easier to understand)
  - **equal-frequency binning** (bins have variable width; not covered in the lecture, described in the textbook; aka equal-depth binning: used in databases for fast data access)

Number of Bins

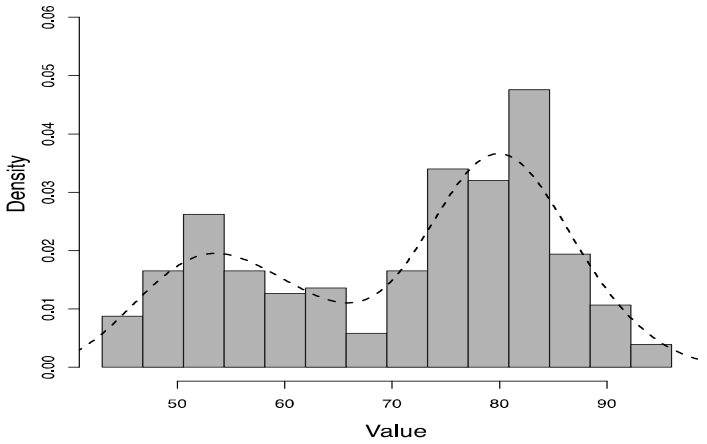
- **Need to manually provide the number of bins (default is 10 bins)**
- Deciding the number of bins can be difficult; the bin length (eg range) should make sense for the target problem/domain

- Trade-offs:
  - If number of bins is very low, we may lose a lot of information (e.g., all data in 1 bin)
  - If number of bins is very high, we may have very few values in each bin or end up with empty bins (e.g., each value in its own bin)

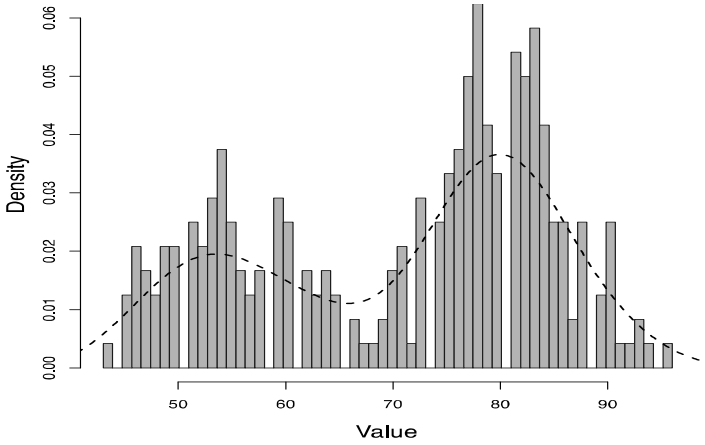
Number of Bins Example



(e) 3 bins



(f) 14 bins



(g) 60 bins

Equal-Width Binning

- **Equal-width binning:** splits feature values into equal sized intervals
- The range of the feature values is split into ***b*** bins each of size ***range/b***
- E.g., for a feature with values ranging in [0,400], split into 10 bins, where each bin is of length 40 (400/10); first bin corresponds to [0,40), second bin to [40, 80), …, last bin [360, 400], etc.
- **We also discussed this when we covered histogram plots**

Sampling of instances = working with different subsets of rows (applied to examples/rows)

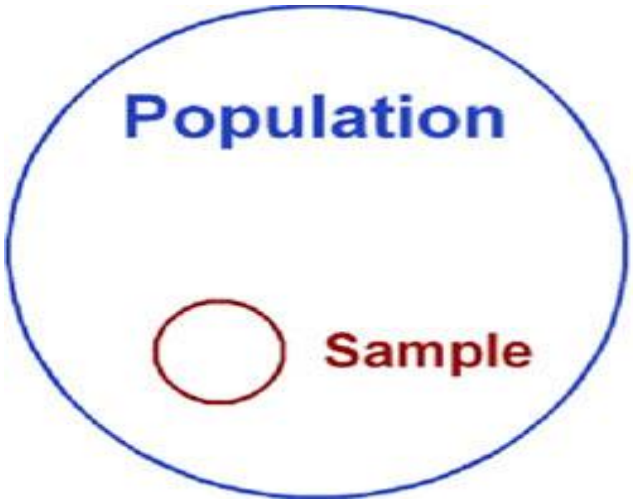
Sampling

- When the dataset is very large, we might not use the full dataset but instead **sample** a smaller percentage of rows; our data is usually already a sample from some population

- When sampling, we need to ensure that resulting datasets are representative of the original data (e.g., no unintended **bias** is introduced during sampling; example bias: the sample only has rows with Age < 20)

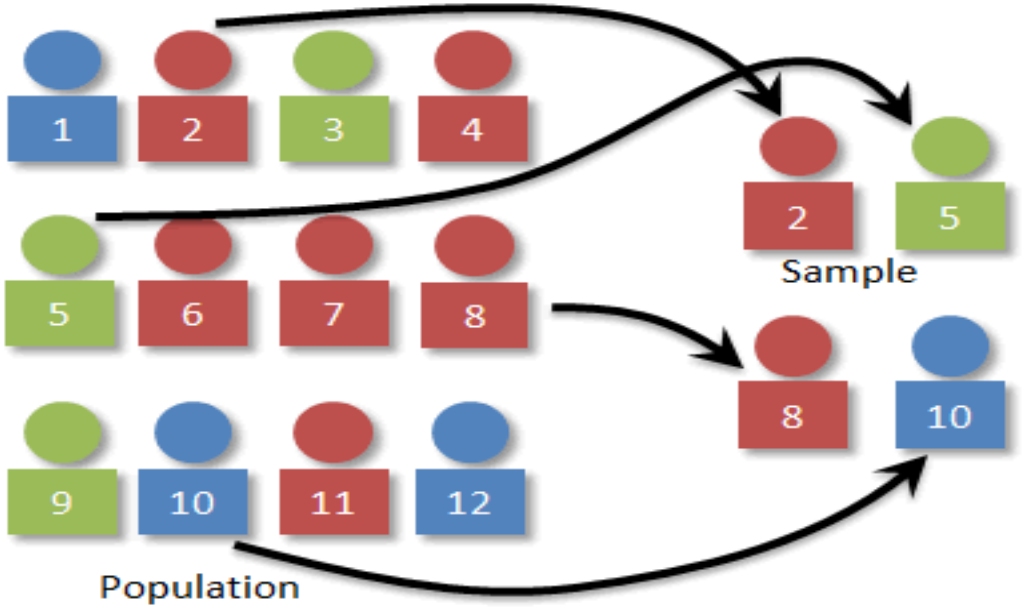
- Common forms of sampling include:
  - **random sampling**
  - **stratified sampling**
  - **over-sampling**

  (**under-sampling)**



Random Sampling

- Randomly select a proportion ***s%*** of the instances from a large dataset to create a smaller set (eg 100k sample from 1 million rows dataset is a 10% sample)
- Random sampling is a good choice in most cases as the random nature of the selection of instances should avoid introducing bias



Source: https://faculty.elgin.edu/dkernler/statistics/ch01/1-4.html

Stratified Sampling

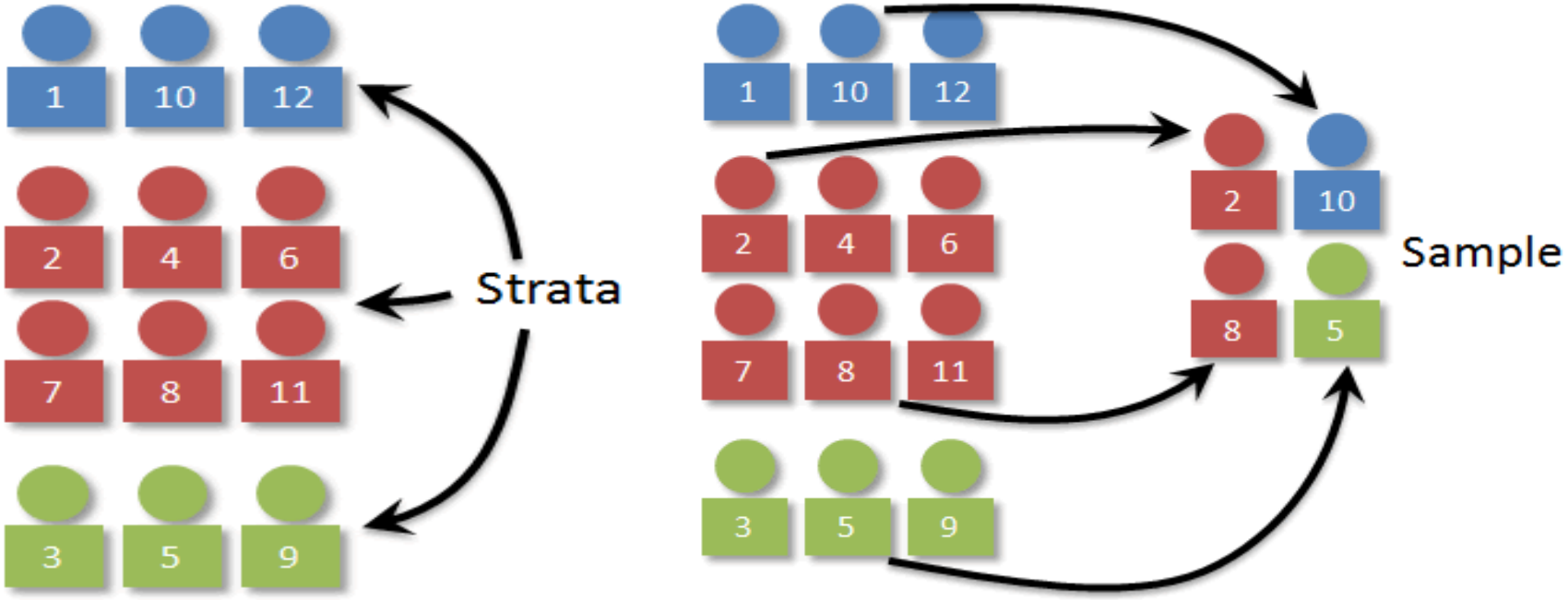
- In some cases, random sampling is not appropriate, e.g., when some levels of a feature are under-represented (very few rows have that value for the feature, e.g., unbalanced groups or classes)

- If we use random sampling our sample may completely miss some of the levels of the feature; those levels will not be represented at all in our sample

- **Stratified sampling** ensures that the relative frequencies of the levels of a specific **stratification feature** are maintained in the sample

Stratified Sampling

- To perform stratified sampling:
  - Divide rows into groups of rows that have a particular level for the **stratification feature**
  - Apply random sampling to each group. Collect subsamples into stratified sample



Stratified Sampling: An Example

- **Population:** 1,000 people

- **Population Strata** (grouped by Height):
  - **Short:** 10% of population (100 people)
  - **Medium:** 70% of population (700 people)
  - **Tall:** 20% of population (200 people)
- **We want a 10% sample to work with:** 10% of population = 100 people sample

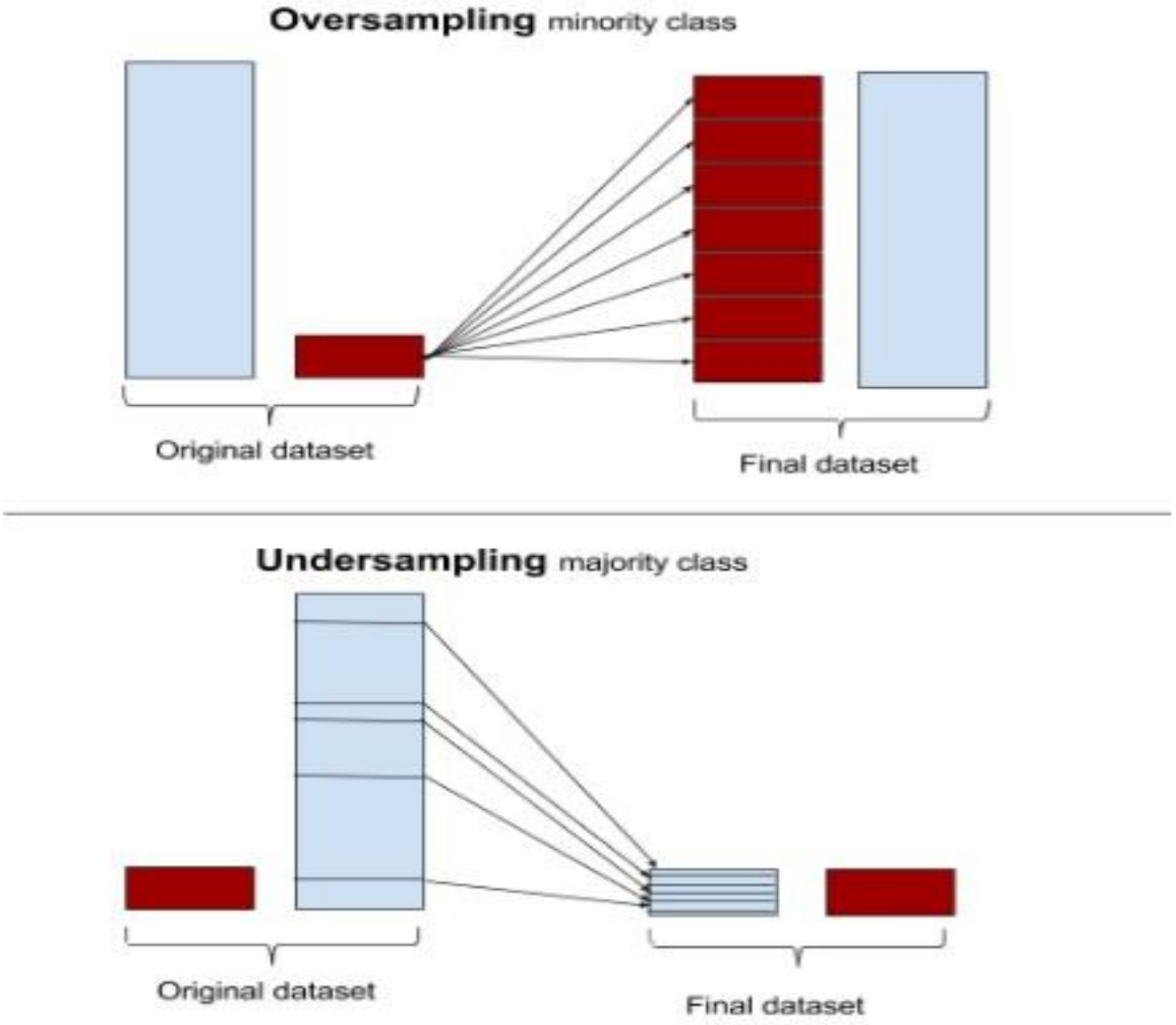
- **Random 10% Sample** of 100 people: (3 Short, 80 Medium, 17 Tall)
- **Stratified 10% Sample** of 100 people: (10 Short, 70 Medium, 20 Tall)
- **Steps to create stratified sample:**
  - Group rows by feature level (strata), we have 3 groups of rows (Short 100 rows, Medium 700 rows, Tall 200 rows)
  - Random Sample of 10% from each group: 10% Short = 10, 10% Medium = 70, 10% Tall = 20.
  - Assemble the small samples into Stratified Sample of 100 people (10 Short, 70 Medium, 20 Tall). Follows the original proportion of strata in the population. Avoids over or under-representation of some strata.

Over/Under Sampling

- In contrast to stratified sampling, sometimes we want a sample to contain different relative frequencies of the levels/strata of a particular feature than in the original dataset

- E.g., we want to build a sample where the target feature **fraud** and **not-fraud** is equally represented (aka **balanced sample** with same number of rows for fraud/not-fraud)

- To do this, we can use **over-sampling** (or under-sampling).
  - This is **only done on the training set. The test set is left unbalanced, to represent the true data distribution.**



Source: http://svds.com/learning-imbalanced-classes/

Over-Sampling

- Example: Create a **balanced training sample** by using all the **not-fraud (majority class)** instances and an equal size sample of **fraud (minority class)** instances

- This is usually done by **data augmentation**: create new synthetic samples by adding some noise to the original fraud samples, i.e., create close-duplicate rows by slightly altering the feature values

- Many common sampling strategies and Python packages, e.g., sklearn.utils.resample or SMOTE.

Over-Sampling

- After dividing the dataset into groups (eg classes), the number of instances in the *largest* group becomes the over-sampling target size; from each smaller group, create a sample containing that number of instances using **data augmentation**

- These larger samples are combined to form the overall over-sampled dataset. **The resulting dataset is called balanced** (i.e., equal representation for each class).

- Can also use random sampling with replacement to oversample. This can be problematic for evaluation and needs to be done only on the training data (keep duplicates away from test data; keep test data realistic for the problem domain)

Summary

- **Data preparation** changes the data to be used for subsequent analysis and training of predictive models

- The original input feature is dropped and replaced with the new feature (which was scaled by normalization or binning)
- Keep track of changes done to original data (i.e., record changes in the data quality plan)
- Need to apply similar feature transformations to new (test) data (otherwise feature scales are not comparable)
- Although sampling may help to balance the **training set**, the **test set should follow the real distribution** to avoid being over-optimistic about our results; **always test on realistic data for the target problem!**

Lab5

- Small group discussion on lecture concepts, lab material, code (DQR, DQP, cleaning, plots, etc), project work

- Example code for data preparation discussed in this lecture: Lab5-Notebook-DataPrep-BasketballABT-Example_Normalisation-Standardisation-Binning-Sampling.ipynb

References

- **FMLPDA Book: Fundamentals of Machine Learning for Predictive Data Analytics**, John D. Kelleher, Brian Mac Namee, Aoife D'Arcy, MIT Press, 2015 (http://machinelearningbook.com/)

- Good short notes, examples and visualisations for key aspects of Data Analytics: https://e2eml.school/blog.html#101

- **Scaling, binning, sampling:**

- https://www.analyticsvidhya.com/blog/2020/04/feature-scaling-machine-learning-normalization-standardization/
- https://machinelearningmastery.com/robust-scaler-transforms-for-machine-learning/
- http://www.marcoaltini.com/blog/dealing-with-imbalanced-data-undersampling-oversampling-and-proper-cross-validation
- http://svds.com/learning-imbalanced-classes/
- http://www.bbc.co.uk/schools/gcsebitesize/maths/statistics/samplinghirev3.shtml
- https://stats.stackexchange.com/questions/798/calculating-optimal-number-of-bins-in-a-histogram
- https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/